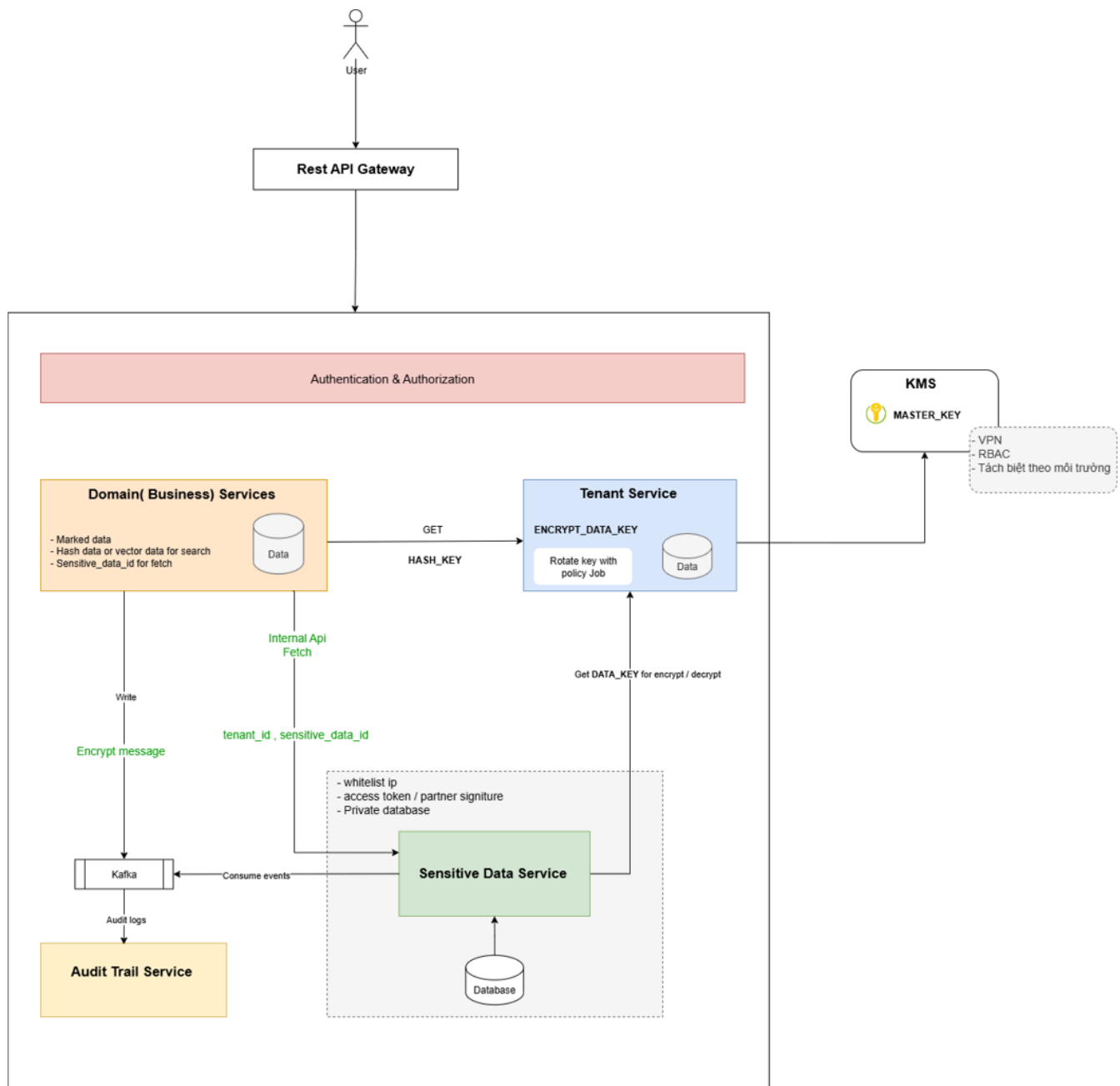


Sensitive Data Service

- **Sensitive Data Service (SDS)** là lớp bảo mật trung tâm của hệ thống Fulfillment SaaS O2O, đảm bảo tất cả dữ liệu nhạy cảm (PII/KYC) được mã hóa, tách biệt, và truy cập theo quyền.
- **Bảo vệ tuyệt đối dữ liệu nhạy cảm (PII)**, đảm bảo các tiêu chí bảo mật thông tin GDPR, PCI DSS, ISO 27001 : không lưu trữ plaintext của các trường như số điện thoại, địa chỉ, email, số CMND,... trong bất kỳ service hoặc DB nào.
- **Cô lập theo tenant** : Dữ liệu nhạy cảm giữa các tenant phải hoàn toàn cách ly, ngay cả khi bị lộ một tenant key.
- **Hạn chế quyền truy cập (RBAC)**
- **Defense-in-Depth** : Ngay cả khi DB hoặc message queue bị lộ, attacker không thể đọc PII
- **Sẵn sàng tuân thủ GDPR , PDPA** : Hệ thống có khả năng thực hiện yêu cầu “**data delete**” hoặc “**anonymize**” theo quy định
- **Audit & Non-repudiation** : Mọi hành vi đọc/ghi/decrypt dữ liệu KYC phải được log lại để phục vụ kiểm tra bảo mật hoặc pháp lý
- **Service communicate** : Định nghĩa cách giao tiếp giữa các service , cách lưu trữ thông tin và luồng xử lý
- **Risk management** : Giải pháp quản lý rủi ro khi có các sự cố về bảo mật không mong muốn
- **Rotate key policy** : có thể rotate key định kỳ mà không bị gián đoạn dịch vụ
- **Scalability**: đảm bảo khả năng đọc ghi scale theo số lượng tenant , scale ra global đa quốc gia

Kiến trúc tổng thể



[O2O_Hasaki-KYC.jpg](#)

- Background job thực hiện re-encrypt incremental theo batch để tránh gián đoạn

• HKDF-SHA256

- Theo chuẩn RFC 5869, NIST SP 800-56C
- Hash với secret key , salt , info + counter

- **AES 256 + GCM (Galois/Counter Mode)**

- Dùng làm thuật toán mã hóa dữ liệu
- Chọn chuẩn GCM vừa mã hóa vừa xác thực (AEAD) → chống được giả mạo, thay đổi nội dung .

- **Tenant-level Key / Data Key**

- Sinh data key/tenant key bằng HKDF-SHA256
- Mỗi tenant có data key riêng, được mã hóa bằng **AES 256** với **MASTER_KEY**
- Khi tenant bị lộ thì không ảnh hưởng đến tenant khác.
- Key mã hóa riêng cho mỗi tenant có thể được rotate định kỳ theo policy hoặc force rotate bởi admin tenant hoặc khi có xác nhận hỗ trợ xử lý sự cố từ phía khách hàng khi bị lộ key
- Việc xoay vòng key cũng tránh được rủi ro lộ tất cả data của tenant mà chỉ ảnh hưởng đến dữ liệu đã được mã hóa bởi key bị lộ

- **Internal Service (dùng cho nội bộ)**

- Internal Service muốn truy cập phải được Sensitive Service đăng ký và cung cấp key : Partner Id và Serect Key

- **Private Network**

- Chỉ những service nội bộ mới được call grpc/http vào hệ thống lưu trữ data nhạy cảm
- Database chỉ Sensitive Service mới được phép truy cập
- Các Internal Server nội bộ muốn truy cập lấy thông tin từ Sensitive Service cũng phải thông qua 1 gateway và được filter bằng cách authen với Partner Key và Serect Key

- **Masked Data + Hashed Suffix , Search by Encrypted Data**

- Masking để hiển thị UI an toàn.
- Hash suffix + salt theo tenant_id → hỗ trợ search mà không cần plaintext.
 - Hỗ trợ search 1-4 số cuối nhanh mà không cần truy cập thông tin chính
- Được sử dụng ở service dùng đến thông tin nhạy cảm như Order, Purchase ...
- Search
 - Search chính xác không search tương đối tránh scan data

- Hash-based search index cho tìm kiếm bằng hash data với hash full thông tin hoặc 2-3-4 số điện thoại, CMND, CCCD, Bank Id Card
- Vector search for address, name ..

• RBAC

- Chỉ **Tenant Admin** có quyền decrypt dữ liệu.
- Phân quyền trên từng nghiệp vụ trước khi truy cập vào dữ liệu nhạy cảm.
- Chỉ cung cấp thông tin khi có danh sách kyc_id cụ thể (thông tin này có được nếu được phân quyền từ giao dịch), không implement các api như get list, search tương đối tránh trường hợp rò rỉ data nếu bị scan
- Phân quyền trên từng field dữ liệu
- Phân quyền cho từng service có thể truy cập vào từng kiểu dữ liệu thông qua IP whitelist
- Dữ liệu được push vào kafka cũng phải được phân quyền trên từng topic thông qua ACL

• Audit Trail + Kafka Events

- Log lại tất cả hành động quan trọng.
- Kafka event giúp **CustomerService / SupplierService** cập nhật trạng thái tức thì.
- Các event liên quan đến thông tin nhạy cảm cũng cần được tách biệt, và mã hóa payload
- Audit log chỉ chứa hash hoặc mark data, không lưu ciphertext hay plaintext

• Risk Management

- Master key được quản lý bởi KMS (**Vault**), được phân quyền và VPN nghiêm ngặt nên tỉ lệ lộ master key rất ít, trường hợp lộ key master sẽ có job chạy lại dữ liệu cho tất cả tenant mà không cần thay đổi thông tin về bảo mật
- Nếu khách hàng hoặc hệ thống bị lộ key mà 1 tenant nắm giữ thì chỉ bị ảnh hưởng ở version key hiện tại của 1 tenant đó không ảnh hưởng đến hệ thống chung → vẫn có cơ chế chạy lại thông tin bảo mật đảm bảo xử lý sự cố
- Dữ liệu nhạy cảm được lưu trữ tách biệt so với key mã hóa, nên nếu database bị tấn công, thì cũng chỉ là thông tin đã được mã hóa hoàn toàn không thể xem được nếu không có master key, IV key và key mã hóa.
- Không phân tán dữ liệu mã hóa ở nhiều nơi giảm rủi ro rò rỉ
- Nếu database bị tấn công hoặc sao chép sẽ không thể nào xem được thông tin nếu không có đủ master_key, data_key

• Q&A

- Tôi chưa hiểu lắm về các loại key mà bạn nói , giải thích model key cho tôi hiểu hơn ?

```
1 MASTER_KEY (in KMS)
2 |
3 | ─ PLAIN_TENANT_KEY
4 |   | ─ Hash Data → HASH_KEY (PLAIN_TENANT_KEY,internal_salt, info)
5 |   | ─ Encrypt Data Key → DATA_KEY (PLAIN_TENANT_KEY,internal_salt, info)
6 |   | ─ Kafka Event → EVENT_MESSAGE_KEY (PLAIN_TENANT_KEY,internal_salt, info)
7 |
8 | ─ Protection:
9 |   - Key isolation
10 |   - Rotation
11 |   - Audit
12 |   - Zero-trust design
13
```

Key	Mô tả	Sinh ra	Lưu trữ	Cách bảo vệ
MASTER_KEY	Gốc hệ thống mã hóa dùng mã hóa các loại key	Tạo trong KMS/Vault (AES-256).	KMS	-Không lưu trong code, config, chỉ lưu secret bất kỳ đâu cũng không view được trừ service tenant -Audit nơi sử dụng
PLAIN_TENANT_KEY	Key gốc cho từng tenant hoặc nghiệp vụ	Random AES-256 key	Không lưu dạng plain text	-chỉ sinh ra lúc tạo mới tenant , hoặc khi có policy chạy xoay vòng key
TENANT_KEY	Mã hóa dữ liệu nhạy cảm của từng tenant.	HKDF từ PLAIN_TENANT_KEY (PLAIN_TENANT_KEY,internal_salt, "encrypt_data_v1")	Database riêng cho Tenant Service	-Chỉ Tenant Service lưu vào database riêng -audit tất cả hoạt động yêu cầu key bằng Api -Không lưu trữ memory/cache quá lâu
HASH_KEY	Key dùng hash dữ liệu search	HKDF từ PLAIN_TENANT_KEY	Không lưu , sinh ra khi cần	- lưu độc lập bằng database riêng , cô lập với các database khác

		(PLAIN_TENANT_KEY, internal_salt, "hash_search_data_v1")		
EVENT_MESSAGE_KEY	Key dùng mã hóa dữ liệu push vào message queue (kafka, redis , rabbitMq)	HKDF từ PLAIN_TENANT_KEY (PLAIN_TENANT_KEY, internal_salt, "encrypt_event_message")	Không lưu , sinh ra khi cần	

- Làm sao để bảo vệ MASTER_KEY hiệu quả hơn ?
 -
- Vì sao cần đến tenant_key/data_key thay vì chỉ dùng 1 master_key cho đơn giản ?
 - Tenant key hoặc data key
 - Nếu key bị lộ (qua log, dev, backup), mọi tenant đều bị giải mã được.
 - Không thể rotate, nếu cần chạy lại dữ liệu phải chạy lại cho tất cả dữ liệu chi phí rất cao
 - Không tuân thủ GDPR/ISO27001/PCI DSS (dữ liệu từng tenant phải độc lập , cần thiết khi xóa account , xóa dữ liệu ...)
 - Tập trung thì tăng rủi ro như là bỏ tất cả trứng vào 1 rổ
 - Các hệ thống lớn (AWS, Stripe, Shopify, Twilio, GCP) đều dùng
- Vì sao cần tập trung tất cả data nhạy cảm ở 1 Service thay vì rải rác từng nơi ?
 - Giảm rủi ro rò rỉ thông tin cá nhân (PII leak) : Khi dữ liệu nhạy cảm nằm rải rác ở nhiều service (Order, Customer, Payment...), mỗi nơi đều có nguy cơ leak → tổng rủi ro gấp N lần. Việc tập trung vào **1 service duy nhất** giúp kiểm soát điểm truy cập, audit, và bảo vệ thống nhất.
 - Quy định bắt buộc phải có cơ chế **audit, mask, revoke access, log đọc/ghi** với dữ liệu cá nhân. Nếu dữ liệu phân tán việc này thực hiện rất khó khăn
 - Giảm bề mặt tấn công : key chỉ lưu một nơi , data chỉ 1 service được truy cập, vpn cũng độc lập tránh phải cấu hình key ở nhiều nơi
 - Centralized Audit & Monitoring : Dễ kiểm soát , theo dõi & xử lý sự cố

- SDS được thiết kế như một domain riêng biệt, tách biệt về hạ tầng, key, database, RBAC và network (chỉ cho phép internal call). Các domain khác chỉ truy cập thông qua `sensitive_data_id`
- Việc xóa hoặc ẩn thông tin cá nhân khi người dùng yêu cầu có thể thực hiện tập trung, thay vì phải chạy trên toàn hệ thống.
- Nếu mỗi service tự triển khai, code duplication và lỗi bảo mật là không tránh khỏi có thể tăng thời gian phát triển
- **Vì sao cần xoay vòng key ?**
 - Giảm thiểu rủi ro nếu key bị lộ
 - Tuân thủ quy định bảo mật (**GDPR, PCI DSS, ISO27001**)
 - Ngăn key reuse quá lâu
 - Khi tenant ngừng sử dụng dịch vụ hoặc bị chấm dứt hợp đồng, có thể **revoke tenant_key** để khóa hoàn toàn khả năng giải mã dữ liệu của tenant đó
- **Vì sao dùng AES 256 GCM để mã hóa dữ liệu ?**
 - Dùng **AES-256-GCM** vì đây là chuẩn mã hóa mạnh nhất hiện nay, được các tổ chức như **NSA, NIST** khuyến nghị dùng để bảo vệ dữ liệu nhạy cảm.
 - Nó không chỉ mã hóa dữ liệu mà còn kiểm tra tính toàn vẹn (integrity), giúp đảm bảo dữ liệu không bị thay đổi hay giả mạo.
 - Dùng trong TLS 1.3, OpenSSL, AWS KMS, Google Cloud KMS, PostgreSQL TDE
 - AWS/GCP KMS hỗ trợ native AES-GCM, dễ rotate và kiểm soát lifecycle key nếu cần thay đổi môi trường infra
- **Vì sao dùng HKDF - 256 để hash dữ liệu ?**
 - HKDF giống như có **một công thức riêng cho mỗi nhóm dữ liệu**, dù ai đó biết cách bẻ, họ cũng không thể đoán ra kết quả nếu không có “muối bí mật” (salt).
 - Nó giúp bảo vệ dữ liệu như việc **không dùng cùng mật khẩu cho tất cả tài khoản** — mỗi tenant, mỗi domain có mã riêng của họ.
- **Vì sao không dùng SHA 256**
 - SHA-256 an toàn về mặt lý thuyết, nhưng không đủ an toàn trong thực tế khi dùng cho dữ liệu nhạy cảm, vì nó không có salt và cho ra cùng kết quả với cùng dữ liệu, nên dễ bị tấn công đoán ngược
- **Nếu không phải multi-tenant thì có dùng dạng data_key theo giải pháp này được không?**
 - Dù hệ thống chỉ có một tenant, việc dùng **data key riêng** giúp chia nhỏ và bảo vệ dữ liệu tốt hơn.

- Nếu dùng một khóa chung, khi khóa đó bị lộ là toàn bộ dữ liệu bị ảnh hưởng. Còn khi tách ra thành nhiều data key, rủi ro được giới hạn — chỉ một phần nhỏ dữ liệu bị ảnh hưởng, dễ thay thế hoặc xoay khóa mới mà không ảnh hưởng toàn hệ thống.
- Ngoài ra, cách này cũng giúp chúng ta sẵn sàng nếu sau này mở rộng thành hệ thống nhiều tenant hoặc nhiều sản phẩm
- **Vậy nếu Sensitive Data Service bị down thì hệ thống có bị gián đoạn không?**
 - Nếu SDS bị down, **business write** vẫn hoạt động vì domain sẽ **queue encrypted events** (Kafka).
 - **Read** sẽ bị hạn chế (mask-only) nhưng không block toàn hệ thống

Vai trò tổng thể các microservice

Tenant Service :

- Quản lý key & tenant metadata, Tạo `tenant_encrypt_key` khi tạo tenant mới (mã hóa bằng system MASTER_KEY).
- Tenant Service quản lý key version + status (active/inactive/revoked)
- Lưu trữ: Bảng `tenant_keys(tenant_id, key_version, cipher_key, status, created_at, rotate_at )`.

Sensitive Data Service

- Quản lý toàn bộ dữ liệu nhạy cảm của người dùng (KYC/PII).
- Nhận dữ liệu thô từ các service khác (OrderService, SupplierService, v.v).
- Mã hóa PII bằng tenant AES key (AES-256-GCM).
- Giải mã khi cần (chỉ khi user có role hợp lệ).
- Lưu trữ: `kyc_data(id, tenant_id, object_type, object_id, cipher_data, masked_data, suffix_hash, key_version, created_at, modified_at, status_id)`.
- Database được tách biệt hoàn toàn , backup với chế độ thường xuyên hơn

Domain Business Service

Bao gồm tất cả các service hoặc domain chứa nghiệp vụ liên quan đến thông tin nhạy cảm (Customer, Supplier, User , Order , Purchase , Shipping ...)

- Mask dữ liệu để hiển thị trong UI.
- Tạo hashed suffix (search index).

- Hỗ trợ search an toàn (theo hash, suffix hash).

Customer Service

- Quản lý profile khách hàng (liên kết với KYC).
- Chỉ lưu thông tin không nhạy cảm: `customer_id, tenant_id, kyc_id, customer_type`.
 - Khi cần dữ liệu KYC → gọi `Sensitive Data Service`.

Order Service

- Xử lý đơn hàng, không giữ dữ liệu nhạy cảm.
- Khi cần KYC (ví dụ shipping address, phone) → query sang `CustomerService` → `Sensitive Data Service`.
 - Chỉ hiển thị masked data trong màn hình quản trị.
- Lưu trữ: `order(id, tenant_id, customer_id, product_id, masked_name, masked_phone, status, created_at)`.

AuthService (IAM)

- Quản lý identity & access.
- RBAC (role-based access control): Admin, Operator, Viewer.
- Giới hạn quyền giải mã PII chỉ cho user role nhất định.
- Xuất JWT token để các service validate.

Audit Service

- Ghi lại tất cả hành vi liên quan đến PII.
- Append-only audit log (không xóa).
- Mọi action liên quan PII (create, update, decrypt) đều ghi log.
- Kafka consumer → lưu log vào MongoDB (search by action , key or index search , full text search, vector search ...)
- Lưu trữ: `audit_log(id, actor_id, tenant_id, action_id, function_id, target_id, service_name, timestamp, metadata)`.
- Dữ liệu audit chỉ chứa thông tin hash hoặc mark data

API Gateway (gRPC Gateway + Ingress)

- Entry point cho toàn bộ request.
- Validate JWT.
- Route request đến service tương ứng.
- Throttle / rate limit để bảo vệ service KYC.

Database Structure

tenant_crypto_keys

- tenant key dùng quản lý riêng key mã hóa cho từng tenant , đảm bảo nguyên tắc độc lập dữ liệu theo qui định
- Hệ thống không quản lý theo tenant vẫn có thể sinh key tương tự , có thể chia phạm vi theo domain

id	tenant_id	cipher_key_data	key_version	status
1124780565	123456	U2FsdGVkX18J+tcjXEv3YteUxGp70It9cJGIFexBmDmFmH82VzTp8GfIAe3O8CPW	1760171371	ACTIVE
	123456	U2FsdGVkX18J+tcjXEv3YteUxGp70It9cJGIFexBmDmFmH82VzTp8GfIAe3O8CPW	1760171372	CURRENT

- nếu tenant key / datakey bị lộ thì chỉ ảnh hưởng đến dữ liệu của riêng nó

sensitive_data

sid	tenant_id	object_type	object_id	cipher_data	suffix_hash	full_hash	key_version
#	1234	customer	12345	U2FsdGVkX1+5AdDFoMsBm0MPqPQh4rJqKSeS5RY/LlaccIOzpES OQdkNFRYSNTZz	AdDFoMsBm0MPqPQh4rJqKSeS	dDFoMsBm0MPqPQh4rJqKSeS5RY/LlaccIOzpES pESOQdkNFR	1760171372
#	1234	order	12453	U2FsdGVkX18lb+QnAXW2WudoWunRvWH7rfn6XQO7	2FsdGVkX18lb+QnAXW2WudoWunRvWH7rf	udoWunRvWH7rfn6XQO7RhvsiafMcW2uf8H	1760171372

				RhvsiafMc W2uf8H6x sSpojNrqtS d4dnm7so agwKK6F4 6jw==		6xsSpoj NrqtS	
--	--	--	--	--	--	------------------	--

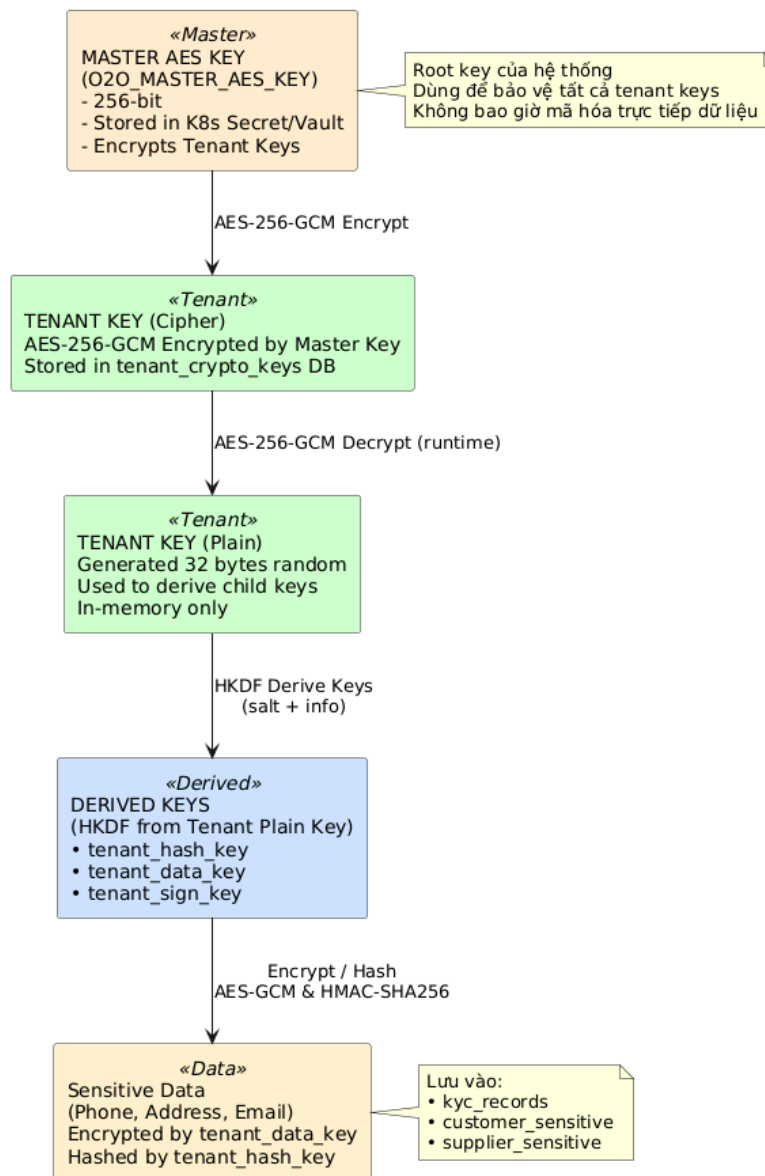
orders / customers / supplier / object sensitive info

id	tenant_id	kyc_id	suffix_hash	full_hash	key_version
1977370192	12453	197737019	FsdGVkX18lb+QnAXW2WudoWunRvWH	2FsdGVkX18lb+QnAXW2WudoWunRvWH7rf	1760171372

SƠ ĐỒ LƯỒNG XỬ LÝ

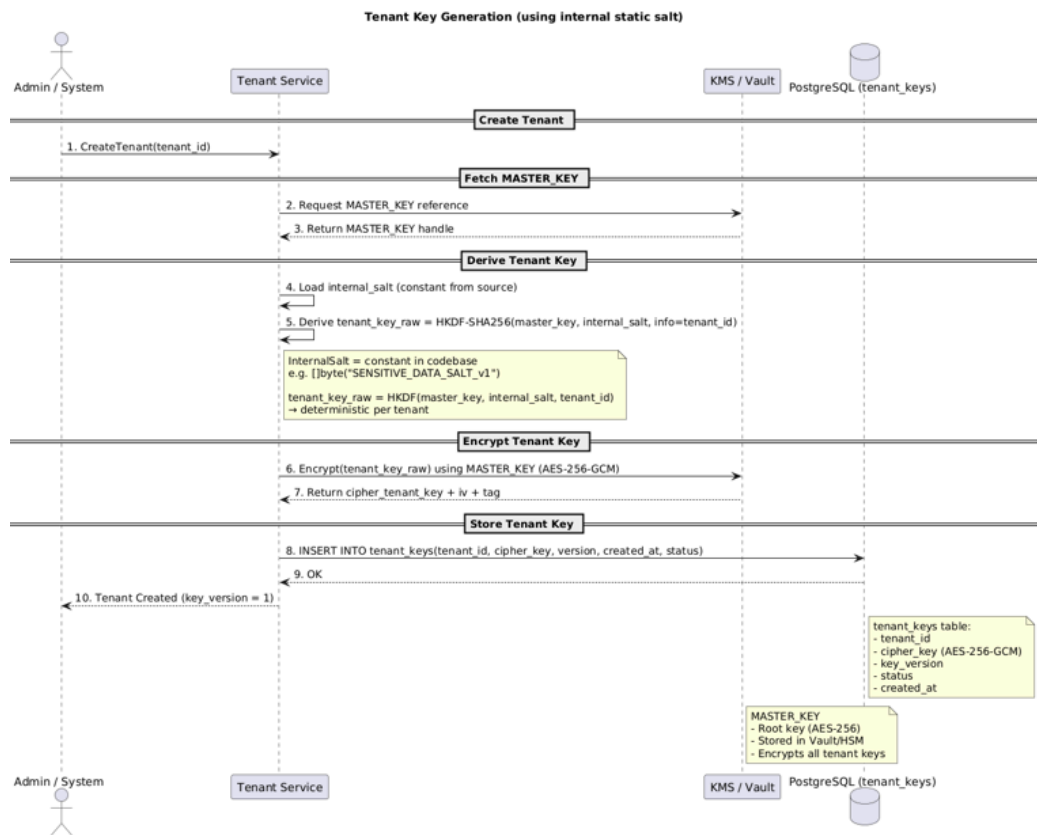
Key Model

O2O Sensitive Data Service - Key Hierarchy Model

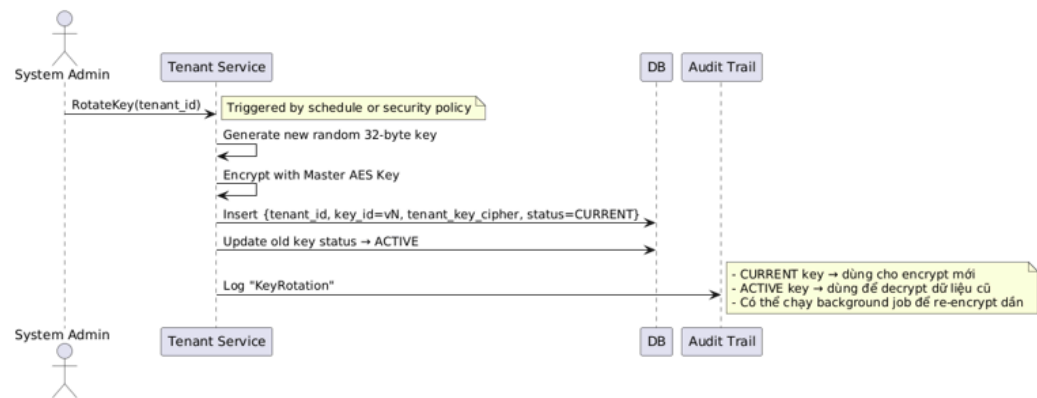


```
1 func DeriveTenantKey(masterKey, salt []byte, info string) []byte {
2     hkdf := hkdf.New(sha256.New, masterKey, salt, []byte(info))
3     derivedKey := make([]byte, 32) // AES-256 key size
4     if _, err := io.ReadFull(hkdf, derivedKey); err != nil {
5         panic(err)
6     }
7     return derivedKey
8 }
```

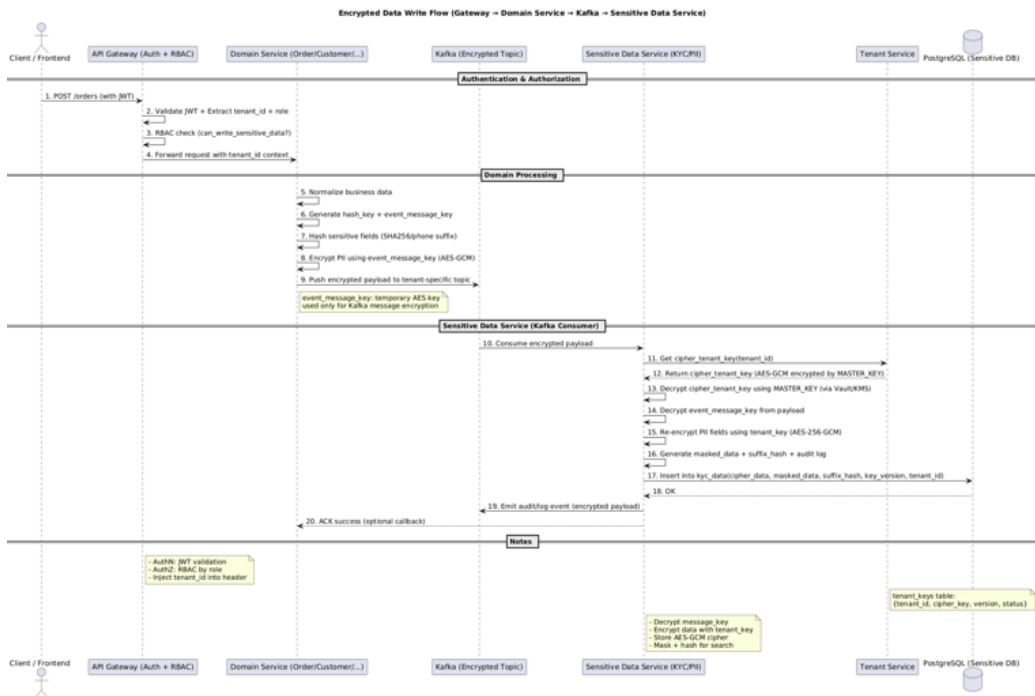
Create Data Key



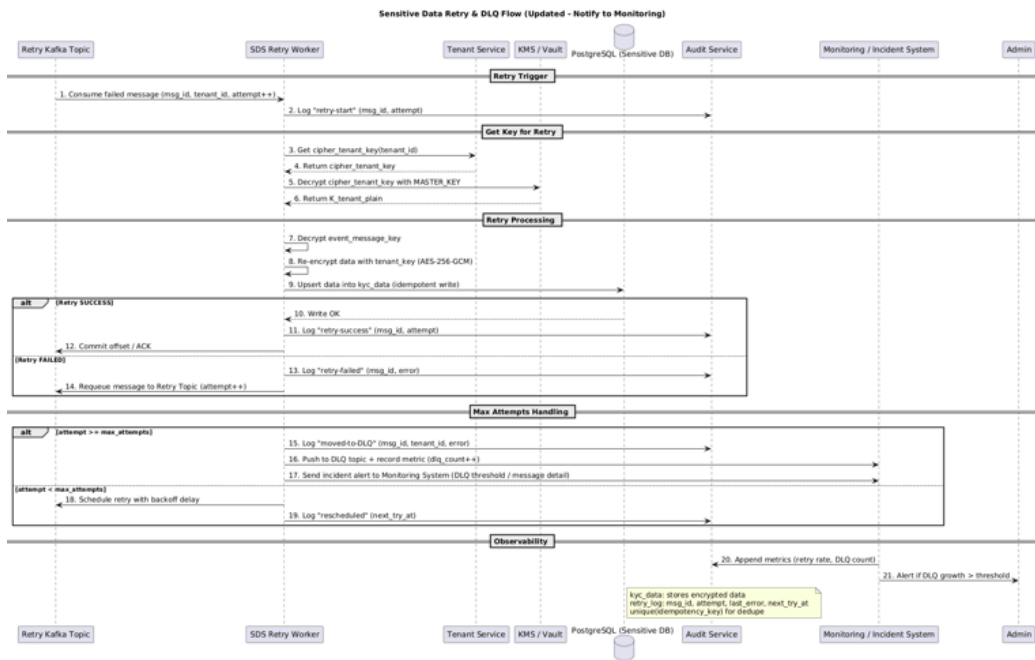
Rotation Data Key Flow



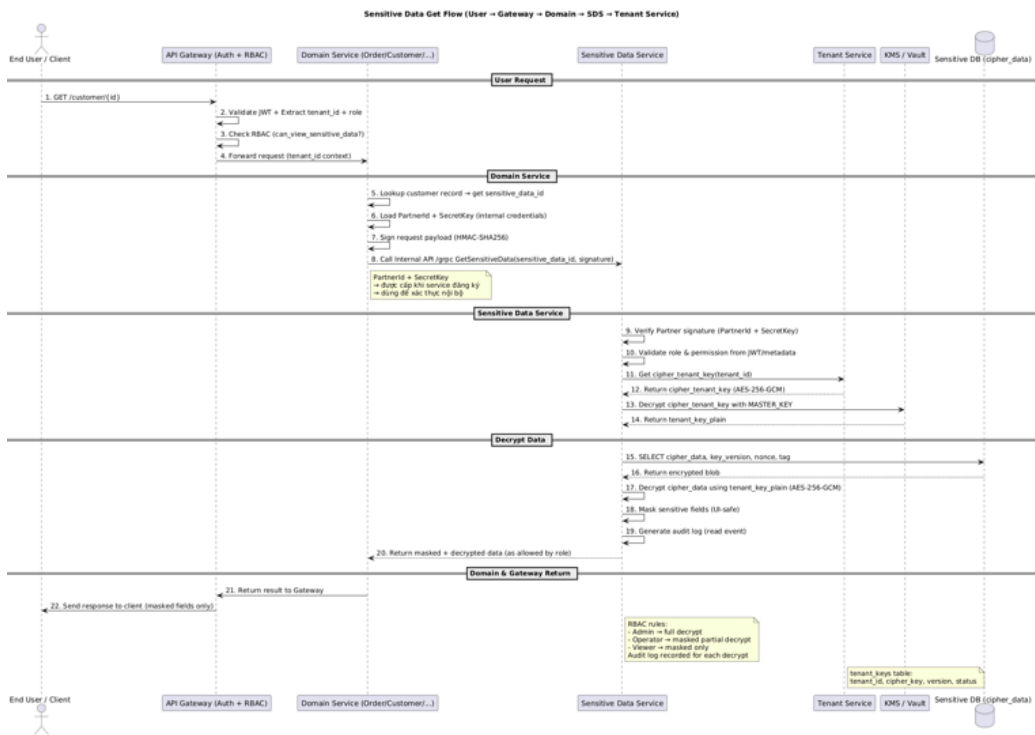
Decrypt PII Flow



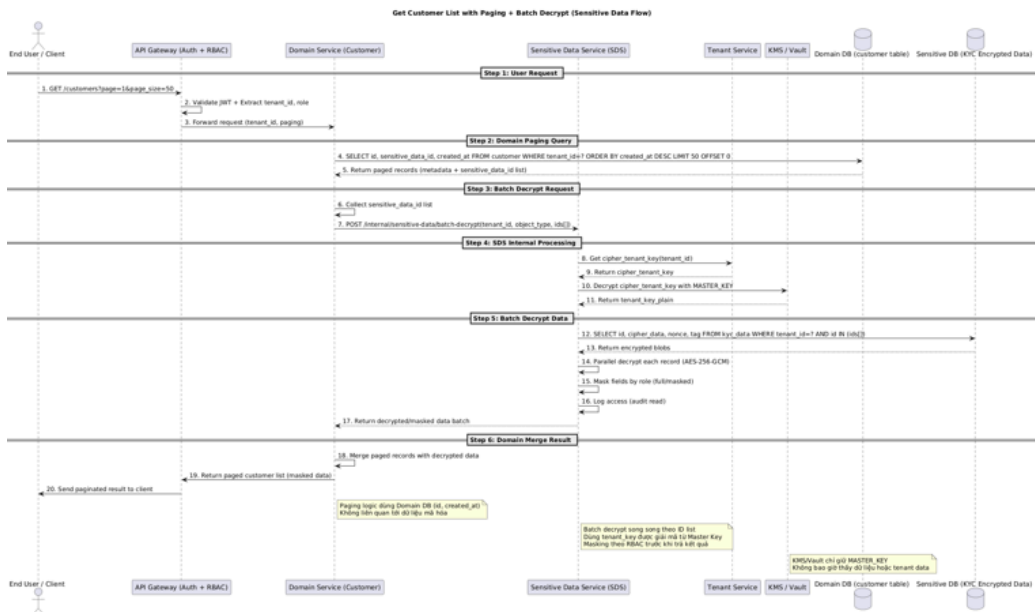
Retry Encrypt Data Flow



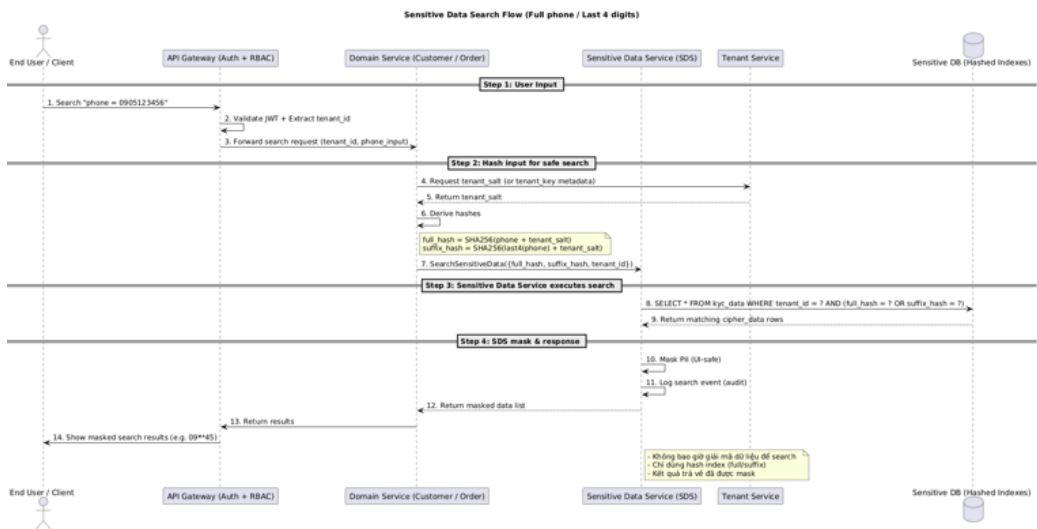
Fetch Entity with sensitive_data_id (Order, Customer, Supplier) Flow



Fetch Paging data



Search by Phone Suffix Flow



KỊCH BẢN GIẢI QUYẾT SỰ CỐ

Sự cố xảy ra	Ảnh hưởng	Giải pháp
Master key bị leak	Toàn hệ thống bị ảnh hưởng	-Chỉ cần tạo lại key theo master key mới -Rotate all tenant keys job -Sử dụng backup master key -Thông qua audit truy vết 1 nguồn nào đó yêu cầu để get thông tin nhạy cảm liên tục
key mã hóa dữ liệu theo tenant bị leak	Chỉ ảnh hưởng đến 1 vùng dữ liệu và trên 1 tenant	Rotate key lại cho tenant bị ảnh hưởng , chạy lại job cập nhật dữ liệu trên toàn tenant bị lộ

Lỗi hỏng phân quyền dẫn đến scope truy cập trái phép ?	Chỉ thấy được dữ liệu marked, hash và cipher data	Payload encryption + Kafka ACL
Dev hoặc Ops access DB	1 phần hệ thống	-Sensitive server, database tách biệt độc lập , giới hạn quyền truy cập bằng IP -VPN riêng
Database Tenant chứa data_key bị leak	Tenant Service Chỉ thấy được data đã mã hóa của data_key	-Backup dữ liệu thường xuyên phục hồi đảm bảo dữ liệu
Database nghiệp vụ bị leak	Một vài service	Chỉ thấy được dữ liệu marked, hash
Database sensitive info bị leak	Chỉ thấy được dữ liệu đã mã hóa. Toàn bộ hệ thống liên quan	-Backup dữ liệu thường xuyên phục hồi đảm bảo dữ liệu

TÀI LIỆU THAM KHẢO

[NIST Cybersecurity Framework \(v1.1\)](#)

[ISO/IEC 27001:2022 - Information Security Management](#)

[Google SRE Workbook – Reliability Maturity](#)

OWASP SAMM v2.1 – [The Model](#)

CIS Controls v8 – [The 18 CIS Controls](#)

[AES-GCM and breaking it on nonce reuse](#)

TASKS



**We couldn't find anything matching
your search**

Try again with a different term.